

Assignment 18

Part A:

1. Difference Between while and do-while Loop

The while and do-while loops are both entry-controlled and exit-controlled looping structures in Java, used to execute a block of code repeatedly based on a condition.

while Loop:

- The condition is checked before executing the loop body.
- If the condition is false initially, the loop body will not execute even once.
- It is also called an entry-controlled loop.

Syntax:

```
while(condition) {  
    // code  
}
```

do-while Loop:

- The condition is checked after executing the loop body.
- The loop body executes at least once, even if the condition is false.
- It is also called an exit-controlled loop.

Syntax:

```
do {  
    // code  
} while(condition);
```

2. Difference Between return and System.exit()

Both return and System.exit() are used to stop execution, but they behave differently.

return:

- It is a keyword in Java.
- It terminates the current method.
- It can return a value if the method has a return type.
- Execution continues in the calling method.

System.exit():

- It is a method of the System class.
- It terminates the entire Java Virtual Machine (JVM).

- No further code executes after this call.
- `System.exit(0)` indicates normal termination.

3. Checked and Unchecked Exceptions

Exceptions are unwanted events that occur during program execution.

Checked Exceptions:

- Checked at compile-time.
- Must be handled using try-catch or declared using throws.
- These are subclasses of `Exception` (excluding `RuntimeException`).
- Example: `IOException`, `SQLException`.

Unchecked Exceptions:

- Occur at runtime.
- Not checked at compile-time.
- Subclasses of `RuntimeException`.
- Example: `NullPointerException`, `ArithmeticException`.

4. Difference Between throw and throws

`throw` and `throws` are used in exception handling but serve different purposes.

`throw`:

- Used to explicitly throw an exception.
- Used inside a method body.
- Followed by an object of `Exception` class.

Example: `throw new ArithmeticException();`

`throws`:

- Used to declare exceptions in method signature.
- Informs the caller that the method may throw an exception.
- Followed by exception class names.

Example: `public void readFile() throws IOException`

5. What is a Custom Exception?

A custom exception is a user-defined exception created by extending the `Exception` class or `RuntimeException` class. It is used when built-in exceptions are not sufficient to describe application-specific errors.

Example:

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}
```

Custom exceptions improve code readability and make error handling more meaningful.

Part B:

1. Program to Show Constructor Chaining

Constructor chaining is the process of calling one constructor from another constructor within the same class or from parent class using `this()` or `super()`.

Example Program:

```
class Demo {
    Demo() {
        this(10);
        System.out.println("Default Constructor");
    }

    Demo(int x) {
        System.out.println("Parameterized Constructor: " + x);
    }

    public static void main(String[] args) {
        Demo obj = new Demo();
    }
}
```

2. Program to Demonstrate use of this and super

The 'this' keyword refers to the current class instance variables, while 'super' refers to the parent class members.

Example Program:

```
class Parent {
    int x = 100;
}

class Child extends Parent {
    int x = 200;

    void display(int x) {
        System.out.println("Local variable: " + x);
        System.out.println("Current class variable using this: " + this.x);
        System.out.println("Parent class variable using super: " + super.x);
    }
}
```

```

    }

    public static void main(String[] args) {
        Child obj = new Child();
        obj.display(300);
    }
}

```

3. Program to Count Number of Objects Created Using Static Variable

Static variable is shared among all objects of a class. It can be used to count the number of objects created.

Example Program:

```

class Counter {
    static int count = 0;

    Counter() {
        count++;
        System.out.println("Object created. Count: " + count);
    }

    public static void main(String[] args) {
        Counter obj1 = new Counter();
        Counter obj2 = new Counter();
        Counter obj3 = new Counter();
    }
}

```

4. Program to Demonstrate Static Block Execution Order

Static blocks execute before the main method and are executed only once when the class is loaded.

Example Program:

```

class StaticDemo {
    static {
        System.out.println("Static Block 1");
    }

    static {
        System.out.println("Static Block 2");
    }

    public static void main(String[] args) {

```

```
        System.out.println("Main Method");
    }
}
```

5. Program to Demonstrate Use of Final Variable

The final keyword is used to make a variable constant. Once assigned, its value cannot be changed.

Example Program:

```
class FinalDemo {
    final int MAX = 100;

    void display() {
        System.out.println("Final variable value: " + MAX);
    }

    public static void main(String[] args) {
        FinalDemo obj = new FinalDemo();
        obj.display();
    }
}
```